

-  Secrets
-  ABAP
-  Ansible
-  Apex
-  AzureResourceManager
-  C
-  C#
-  C++
-  CloudFormation
-  COBOL
-  CSS
-  **Dart**
-  Docker
-  Flex
-  Go
-  HTML
-  Java
-  JavaScript
-  JCL
-  Kotlin
-  Kubernetes
-  Objective C
-  PHP
-  PL/I
-  PL/SQL
-  Python
-  RPG
-  Ruby
-  Scala
-  Swift
-  Terraform
-  Text
-  TypeScript
-  T-SQL
-  VB.NET
-  VB6
-  XML



Dart static code analysis

Unique rules to write Clean DART Code

All rules 115

 Bug 15

 Code Smell 100

Tags ▾

Impact ▾

Clean code attribute ▾

Search by name... 🔍

Class names should comply with a naming convention

 Code Smell

Track uses of "TODO" tags

 Code Smell

Deprecated code should be removed

 Code Smell

Cyclomatic Complexity of functions should not be too high

 Code Smell

Constant names should comply with a naming convention

 Code Smell

Unnecessary nullable in final declaration should be removed

 Code Smell

Dependencies should be sorted

 Code Smell

Unused assignments should be removed

 Code Smell

Standard outputs should not be used directly to log anything

 Code Smell

Variables should not be initialized with "null"

 Code Smell

Files should end with a newline

 Code Smell

Dependencies should be sorted

Analyze your code

Consistency - Formatted

Maintainability ⬆

 Code Smell

 Major ⓘ

Dependencies should be sorted in alphabetical ascending order.

Why is this an issue?

How can I fix it?

More Info

Keeping `pubspec.yaml` dependencies in order facilitates lookup and maintenance. There may be multiple criteria to sort dependencies, which make sense in the context of the Dart project on-hand.

However, none of the criteria can be a perfect-fit for all projects. In order to promote consistency across projects, and to follow the [Principle of least astonishment](#), it's best to sort dependencies in alphabetical ascending order.

While it may not be the most optimal sorting criteria for all specific scenarios, it's the most neutral, easily inferrable at first glance, as well as the least surprising.

The rule applies to the following three sections of the `pubspec.yaml` file:

- `dependencies`
- `dev_dependencies`
- `dependency_overrides`

What is the potential impact?

Sorting dependencies in a different order may lead, for example, to duplicated dependencies, where a developer assumed a dependency would appear in a certain section of the `pubspec.yaml` file, while it actually appears in a different section.

Available In:

sonarcloud 

